

作业感受

HW1 挑战内容 5（以下为在 AI 协助下整理的初稿，签名提交前请按自己的真实体会修改）

（1）简述你是如何完成第一次挑战的

我按"跑通 → 测量 → 理解"三步推进。先在无 GPU 的工作机上部署 SGLang 0.5.14 的 CPU 后端，期间借助 AI 助手快速排障：缺依赖就补依赖（openai、compressed-tensors 等），报 No module named vllm 就顺着源码找到 CPU 分支的开关 SGLANG_USE_CPU_ENGINE=1，服务反复被杀就用 cgroup 的 memory.events 定位到 OOM，把 KV 池压到 16384 token 后稳定运行。跑通 /v1/models 与一次推理后，我写了 workload 回放脚本：从 Mooncake trace 采样真实记录、按 input_length 构造 prompt、用指数分布生成泊松到达时刻，并逐请求记录 TTFT/latency。第一版把到达间隔设成 3 秒，系统严重过载、TTFT 全是排队时间，改成 18 秒后指标才反映真实服务能力——这个"先跑通再调负载"的过程让我印象深刻。最后对照源码画流程图、整理对 RadixAttention 的理解。

（2）哪部分最困难，如何克服

最困难的是在没有 GPU 的环境里把推理服务真正跑起来并保持稳定：文档默认 CUDA 环境，按默认参数启动会在初始化阶段被 OOM killer 杀掉，而 free 显示有 100+ GB 可用内存，极具迷惑性。我分三步解决：先确认约束（读 cgroup 的 memory.max / memory.events，发现真实限额只有 8 GB）；再读源码找开关（CPU 后端需要环境变量显式开启）；最后把每项资源（KV 池、并发上限）按限额反推并写进启动脚本。这件事让我体会到：报错信息只是入口，理解系统的资源模型才能根治问题。

（3）从 0 到 1 的科研工作需要哪些方面，对今后科研的启发

- **先建最小可复现系统，再谈优化。**服务跑不通，一切分析都是空谈；复现实验的每一步（版本、参数、数据）都要可追溯。
- **测量先于优化。**先定义指标（TTFT/TPOT/命中率），再设计对照（只改一个变量），结论才有说服力；HW2 的两组负载就是这个思路。
- **诚实记录失败。**第一版 workload 过载、flush_cache 误判等弯路同样有价值，它们往往指向对系统语义的理解偏差。
- **跨层验证。**把"宏观指标—中间观测（cached_tokens）—源码机制"三层对上号，结论才扎实；这也正是本作业两个挑战相互印证的地方。